

ЛР3: Контейнеризация приложения средствами Docker

Студент: Зеленков Евгений

Группа: БР2.2

Тема: контейнеризация микросервисного backend-приложения бронирования ресторанов

Цель работы

Цель работы - упаковать микросервисное приложение в Docker-контейнеры, настроить общий docker-compose.yml и обеспечить сетевое взаимодействие между сервисами.

Состав контейнеров

В приложении используется один общий Node.js проект, но каждый сервис запускается в отдельном контейнере со своим Dockerfile:

Контейнер

Dockerfile

Назначение

api-gateway

docker/api-gateway.Dockerfile
единая входная точка HTTP API
identity-service
docker/identity.Dockerfile
пользователи, регистрация, авторизация
catalog-service
docker/catalog.Dockerfile
рестораны, столы, рейтинг, RabbitMQ consumer
menu-service
docker/menu.Dockerfile
меню и позиции меню
reservation-service
docker/reservation.Dockerfile
бронирования и события статусов
review-service
docker/review.Dockerfile
отзывы и событие review.verified
postgres
официальный образ postgres:16
один контейнер PostgreSQL со всеми БД приложения
rabbitmq
официальный образ rabbitmq:3-management
брокер сообщений RabbitMQ
Также добавлен контейнер seed с профилем seed. Он нужен только для заполнения
базы начальными данными.

Сетевое взаимодействие

Все контейнеры запускаются в одной Docker Compose сети. Сервисы обращаются друг к другу по DNS-именам compose-сервисов:

Переменная

Значение внутри Docker

IDENTITY_SERVICE_URL

http://identity-service:3111

CATALOG_SERVICE_URL

http://catalog-service:3112

MENU_SERVICE_URL

http://menu-service:3113

RESERVATION_SERVICE_URL

http://reservation-service:3114

REVIEW_SERVICE_URL

http://review-service:3115

RABBITMQ_URL

amqp://booking_user:booking_password@rabbitmq:5672

*_DATABASE_URL

postgres://booking_user:booking_password@postgres:5432/...

Внутри Docker не используется localhost для межсервисного взаимодействия. localhost внутри контейнера указывает только на сам контейнер, поэтому используются имена сервисов postgres, rabbitmq, identity-service, catalog-service и другие.

Основной внешний вход в приложение - API Gateway:

http://localhost:3105/api/v1

Остальные backend-сервисы доступны только внутри Docker-сети через expose. Внешний клиент работает через gateway, а прямой доступ к внутренним HTTP-сервисам не открывается.

PostgreSQL и
RabbitMQ
дополнительно
опубликованы на хост
как dev-
инфраструктура:

Компонент	Порт внутри Docker	Порт на хосте
PostgreSQL	5432	5435
RabbitMQ AMQP	5672	5673
RabbitMQ Management UI	15672	15673

Это позволяет запускать сервисы двумя способами: полностью через docker compose up или локально через `npm run dev:*`, используя инфраструктуру из Docker.

В compose для контейнеров оставлены явные `container_name`, чтобы на лабораторной защите было проще читать `docker compose ps` и логи. Для масштабирования через `docker compose up --scale` такие имена лучше убрать, потому что несколько экземпляров одного сервиса не смогут иметь одинаковый `container_name`.

PostgreSQL

По заданию используется один контейнер PostgreSQL. В нем создаются отдельные базы данных для сервисов:

```
identity_db;  
restaurant_catalog_db;  
menu_db;  
reservation_db;  
review_db.
```

Создание БД выполняется скриптом `infra/postgres/init/01-create-databases.sql`, который подключен в compose через `/docker-entrypoint-initdb.d`.

RabbitMQ

RabbitMQ запускается отдельным контейнером. Для сервисов внутри Docker доступны:

AMQP: rabbitmq:5672;

Management UI внутри Docker-сети: rabbitmq:15672;

AMQP с хоста: localhost:5673;

Management UI с хоста: <http://localhost:15673>;

логин: booking_user;

пароль: booking_password.

Catalog-service подключается к RabbitMQ как consumer события review.verified. Review-service и reservation-service публикуют доменные события через RabbitMQ.

Запуск

Из папки labs/lab3/app:

```
docker compose up -d --build
```

После запуска сервисов можно заполнить БД начальными данными:

```
docker compose --profile seed run --rm seed
```

Seed запускается из скомпилированного JavaScript (node dist/seed.js). При локальном запуске с хоста перед ним нужно выполнить:

```
npm run build
```

```
npm run seed
```

Основной API доступен по адресу:

<http://localhost:3105/api/v1>

Проверка состояния контейнеров:

```
docker compose ps
```

Остановка:

```
docker compose down
```

Остановка с удалением данных PostgreSQL и RabbitMQ:

```
docker compose down -v
```

Вывод

В ходе работы микросервисное приложение было контейнеризовано. Для каждого backend-сервиса добавлен отдельный Dockerfile, инфраструктурные зависимости вынесены в контейнеры PostgreSQL и RabbitMQ, а сетевое взаимодействие между сервисами настроено через внутренние DNS-имена Docker Compose.